

Smart Analysis for Field Enforcement Using Deep Neural Network (PPE Detection)

M.G. Nadun Sandeepa
EUSLTC/IS/COM/2020/COM/102
2020/COM/461

Submitted in partial fulfilment of the requirements for the award of the degree
Bachelor of Science in Computer Science [BSc (CS)] to the Department of Computer
Science,
Faculty of Applied Science, Trincomalee Campus, Eastern University, Sri Lanka

Supervisor: Ms. J. Janani

Date of Submission:
17/04/2026

Declaration

I hereby declare that the work presented in this research report is entirely my own. Any information taken from existing literature has been properly acknowledged throughout the document. This work has not been submitted, in full or in part, for any other degree or qualification at this or any other university. I confirm that this document is free from plagiarism, and I understand and accept the consequences should any be found, as determined by the Faculty of Applied Science, Eastern University, Sri Lanka.

.....

M.G. Nadun Sandeepa

EUSLTC/IS/COM/2020/COM/102 | 2020/COM/461

Department of Computer Science

Faculty of Applied Science

Trincomalee Campus, Eastern University, Sri Lanka

Certification of the Supervisors

This is to certify that the research report entitled "**Smart Analysis for Field Enforcement Using Deep Neural Network (PPE Detection)**", submitted by the above-named candidates for the degree of Bachelor of Science in Computer Science, is a record of work carried out by them under our supervision. It has not previously been submitted for any degree, diploma, or other similar award.

We further certify that this report represents the candidates' own independent work.

.....
Signature of Co-Supervisor

Ms. J. Janani

Lecturer

Department of Computer Science, Trincomalee Campus, Eastern University, Sri Lanka

.....
Date

.....
Signature of Supervisor

Ms. J. Janani

Lecturer

Department of Computer Science, Trincomalee Campus, Eastern University, Sri Lanka

.....
Date

Ms. J. Janani

Co-Supervisor

Acknowledgment

Above all, sincere gratitude is expressed to the supervisor, Janani Madam, for her steady guidance, patience, and generosity with her time throughout this project. Her knowledge of deep learning and computer vision gave this work both its direction and its depth, and her encouragement during the harder stages made a genuine difference. It has been a real privilege to work under her supervision.

Thanks are also extended to the Head of the Department of Computer Science, Faculty of Applied Science, Trincomalee Campus, Eastern University, Sri Lanka, and to all the lecturers who contributed to the academic foundation on which this project was built.

Finally, this work would not have been completed without the support of family and close friends. Their patience and encouragement during the busiest periods of the project were invaluable, and that support is deeply appreciated.

Abstract

Every year, workers on construction sites are seriously injured or killed because they are not wearing the right safety gear — things like hard hats, safety vests, gloves, boots, and eye protection. The people responsible for enforcing safety rules simply cannot monitor every worker all the time. They check when they can, but they miss things, and the cost of those missed moments is measured in real harm to real people.

This project built a system that watches construction workers through a camera, works out who is not wearing the right safety gear, recognizes each person by their face, checks whether anyone has moved outside the area they are supposed to stay in, and then speaks to that person by name to warn them. The aim was to create something that a site supervisor could set up and use without needing specialist technical knowledge, and that would run in real time without any noticeable delay.

The system was trained on over twelve thousand images taken from real construction sites, covering ten different situations — both workers wearing the correct gear and workers without it. The face recognition part was tested on fifty people and could correctly identify them almost all of the time, matching each face in under half a second. When tested on a real construction site, the system correctly identified safety issues in nearly nineteen out of every twenty cases, which was a better result than three commercial systems tested under the same conditions.

These results show that this kind of automated monitoring is not just possible — it is reliable enough to make a real difference. Construction sites should not have to depend on a single supervisor to catch every safety violation. A system that watches continuously, knows who it is looking at, and speaks up immediately when something is wrong is a far stronger safety net. That is what this project has built, and the testing confirms that it works.

Keywords: PPE Detection, Deep Learning, YOLO, Facial Recognition, Construction Site Safety

Table of Contents

Declaration.....	2
Certification of the Supervisors	3
Acknowledgment	4
Abstract	5
Table of Contents.....	6
Chapter 01: Introduction	8
1.1 Background	8
1.2 Problem Statement	8
1.3 Objectives of the Study	9
1.4 Significance of the Study	9
1.5 Scope and Limitations.....	10
Chapter 02: Related Work	11
2.1 PPE Detection Using Deep Learning.....	11
2.1.1 Early CNN and Transfer Learning Approaches.....	11
2.1.2 YOLO-Based Real-Time Detection with Multi-Class Support	11
2.1.3 Limitations of Existing PPE Detection Systems.....	12
2.2 Safe Zone Enforcement and Proximity Monitoring.....	12
2.2.1 Hardware-Based Geofencing Approaches	12
2.2.2 Vision-Based Zone Enforcement.....	12
2.3 Facial Recognition for Worker Identification.....	12
2.3.1 Deep Embedding Models for Face Recognition	12
2.3.2 Facial Recognition in Workplace Contexts	13
2.4 Research Gaps and Contribution of the Present Study	13
Chapter 03: Tools and Techniques	15
3.1 Python Programming Language	15
3.2 Google API Integration.....	15
3.3 YOLOv8 — Object Detection	17
3.4 DeepFace and FaceNet	18
3.5 OpenCV	18
3.6 Tkinter.....	18
3.7 pyttsx3 — Text-to-Speech	18
3.8 Google Colab — Training Environment.....	18
3.9 Dataset Tools	18
Chapter 04: Methodology	20
4.1 Research Design.....	20

4.2 Dataset Preparation	20
4.3 Model Training	21
4.4 Face Recognition Database Construction	23
4.5 Safe Zone Definition.....	23
4.6 Real-Time Processing Pipeline	24
4.7 Algorithms and Techniques	26
4.7.1 YOLOv8 Detection.....	26
4.7.2 FaceNet Cosine Similarity Matching.....	26
4.7.3 Polygon Containment Testing.....	26
Chapter 05: Results and Discussion.....	27
5.1 System Output Overview.....	27
5.2 YOLOv8 Training Results.....	27
5.3 Face Recognition Performance	28
5.4 Safe Zone Enforcement Results.....	29
5.5 Real-World Performance Comparison.....	29
5.6 Voice Alert Responsiveness	30
Chapter 06: Conclusion.....	31
Chapter 07: Future Work	32
References.....	33
Appendix.....	35
Appendix A: Research Progress Report	35
Appendix B: Supervisor Report.....	35
Appendix C: Dataset Class Distribution	35
Appendix D: Source Code — Main Application.....	Error! Bookmark not defined.

Chapter 01: Introduction

1.1 Background

Construction sites have always been demanding and dangerous environments. Heavy machinery moves without much warning, working surfaces are often uneven or unstable, and the pace of activity means that workers are regularly exposed to serious physical risks. According to occupational health and safety data, the construction industry records more workplace deaths per worker than almost any other sector, and the majority of those deaths involve failures that could have been prevented.

One of the most consistent and preventable causes of injury is the failure to wear appropriate Personal Protective Equipment (PPE). Helmets, high-visibility vests, gloves, steel-capped boots, and goggles exist precisely because the human body cannot absorb the kinds of forces involved in construction work. Most countries have legal requirements for their use. Even so, non-compliance is common: workers remove equipment during breaks and forget to put it back, wear it incorrectly, or simply do not wear it at all in areas where they believe they are unlikely to be noticed.

The practical problem is straightforward: a site manager responsible for fifty or more workers across a large site cannot realistically observe every individual throughout the working day. By the time a violation is spotted and addressed, the moment of risk may already have passed or worse, may have resulted in injury. What is needed is a system that watches continuously, recognises each worker individually, and responds to violations the moment they occur.

Recent advances in deep learning have made this technically achievable. Object detection models such as those in the YOLO family can identify and locate workers and their equipment within a live video stream in real time. When combined with facial recognition and safe zone boundary logic, such a system can go further than simple detection: it can tell you specifically which worker has removed their helmet, and alert that person by name. This project was built to demonstrate exactly that.

1.2 Problem Statement

At present, PPE compliance on construction sites is enforced through manual supervision. This approach has several well-understood weaknesses: it depends on a supervisor being in the right place at the right time, it cannot scale effectively across large sites, and it provides no mechanism for identifying or alerting specific workers in real time. The following four problems were identified as the focus of this project:

- There is no automated system currently in use at the target site that can detect PPE violations in a live camera feed across all ten relevant compliance classes simultaneously.
- Existing commercial systems can identify that a violation has occurred but cannot identify which individual worker is responsible, which limits accountability and the effectiveness of follow-up.

- Safe zone boundaries — the areas workers must remain within during certain operations are currently enforced only through verbal instruction and manual checking, with no automated detection of breaches.
- No system currently in use can address a specific worker by name in real time at the moment a violation is detected.

Taken together, these gaps mean that PPE compliance is enforced inconsistently, and workers who are paying attention may calculate that violations are unlikely to be noticed outside of formal inspections. An intelligent system that addresses all four problems simultaneously is needed.

1.3 Objectives of the Study

The following objectives were defined for this project:

- To train a YOLOv8l detection model on a custom dataset of 5,140 construction site images, augmented to 12,334 samples, across ten PPE-related classes, achieving a minimum mAP@50 of 90%.
- To build a real-time PPE violation detection pipeline that processes live webcam or CCTV input and identifies workers who are not wearing required equipment.
- To develop a face recognition module using FaceNet with VGGFace2 embeddings, capable of identifying registered workers with a True Positive Rate above 95%.
- To implement an interactive, polygon-based safe zone enforcement system that detects and counts workers who have moved outside defined boundaries.
- To measure and display real-time distances between detected workers, shown directly on the video frame.
- To deliver personalised text-to-speech voice alerts to identified non-compliant workers, with timing controls to prevent repeated flooding of the same alert.
- To provide a simple Tkinter graphical interface that allows non-technical site supervisors to operate the system without specialist support.

1.4 Significance of the Study

This project is significant on two levels. Practically, a system that can reliably detect PPE violations in real time and identify the specific worker involved has the potential to reduce injuries on construction sites in a way that manual supervision cannot. Being addressed by name, immediately, at the moment of a violation is a very different experience from a generic alarm or a supervisor's note at the end of the day — and there is good reason to expect that kind of direct, personal feedback to be more effective at changing behaviour.

Technically, the project brings together capabilities that have each been studied individually but have not previously been combined in a single, open, deployable system: YOLOv8 detection across ten PPE classes, FaceNet-based individual identification, polygon safe zone enforcement, real-time distance measurement, and personalised voice alerts. Python was chosen throughout because it makes this kind of integration straightforward, and the existing Google API connection means the system can be extended to link with wider workforce management platforms when needed.

1.5 Scope and Limitations

This project covers the full development cycle, from dataset preparation and model training through to integrated system deployment and evaluation on a standard workstation with a webcam input. The system operates on a single camera feed and does not currently support multi-camera setups. The face recognition database is pre-built and must be populated manually before deployment. Recognized limitations include reduced detection accuracy in low-light conditions, processing constraints at very high video resolutions, occasional false positive detections when PPE items are only partially visible, and some variation in face recognition accuracy depending on camera angle.

Chapter 02: Related Work

This chapter reviews the existing research relevant to the three main components of the proposed system: PPE detection using deep learning, safe zone and proximity monitoring, and face recognition for worker identification. The chapter ends by identifying the gap that the present research addresses.

2.1 PPE Detection Using Deep Learning

2.1.1 Early CNN and Transfer Learning Approaches

Serious research into automated PPE detection using deep learning began to emerge around 2018 to 2020. Delhi, Sankarlal and Thomas (2020) produced one of the most widely referenced early studies, applying transfer learning to a YOLOv3 backbone to detect hard hats and safety jackets in real time on construction sites [1]. Their system performed well under normal lighting and showed clearly that deep learning could outperform earlier image-processing approaches. Its main limitation was that it only covered two PPE classes and had no way of identifying which individual worker had committed a violation.

Working at roughly the same time, Shao, Li and Zhang (2020) used a modified version of YOLOv3 to detect helmets on construction sites in China [2]. Their work drew attention to a challenge that appears repeatedly in this field: detection accuracy drops noticeably when workers are partially hidden, when the camera angle is steep, or when lighting conditions shift quickly. Finding ways to address these conditions became a consistent theme in the research that followed.

2.1.2 YOLO-Based Real-Time Detection with Multi-Class Support

As the YOLO architecture developed through versions 4, 5, and eventually 8, researchers steadily expanded the number of PPE categories their systems could detect at once. Lee et al. (2023) built a monitoring framework using YOLACT combined with MobileNetV3 for pixel-level detection of PPE items, and added the DeepSORT tracking algorithm to keep track of individuals across multiple frames [3]. This approach achieved a PPE-wearing status accuracy of 91.3%, which was a clear improvement over earlier systems that processed each frame independently. The trade-off was computational cost: running two models simultaneously made real-time deployment more difficult.

Akbarzadeh et al. (2024) carried out a detailed comparison of YOLOv8 against older detection architectures, testing across eight object classes using the CHV and SHEL5K datasets [4]. YOLOv8 produced strong precision and recall results at a model size of only 21 MB, which made it a practical choice for deployment in field conditions. The present study builds directly on this finding, using YOLOv8l as its detection model and extending the class scheme to ten categories — covering both the presence of each PPE item (helmet, vest, gloves, goggles, boots) and its explicit absence (no-helmet, no-vest, no-gloves, no-goggles, no-boots).

2.1.3 Limitations of Existing PPE Detection Systems

Looking across the PPE detection literature, one limitation comes up consistently: most systems detect violations at the scene level. They can tell you that someone in the frame is not wearing a helmet, but they cannot tell you who that person is. Without that identity link, there is no real accountability. Most systems also do not enforce safe zones, do not track distances between workers, and do not send any kind of personalised alert — they detect a problem and leave the response entirely to whoever is watching the screen. The present research was designed specifically to address these gaps.

2.2 Safe Zone Enforcement and Proximity Monitoring

2.2.1 Hardware-Based Geofencing Approaches

Historically, enforcing safe zones on construction sites has been a hardware challenge. From the early 2010s, RFID systems were deployed that attached tags to workers' hard hats and used fixed readers at zone boundaries to track who entered and left [5]. GPS-based geofencing provided a similar function over larger areas. Both approaches had significant drawbacks: they were expensive to install and maintain, difficult to reconfigure when site layouts changed, and vulnerable to signal interference from the steel structures common on construction sites. These problems created a clear need for software-based alternatives.

2.2.2 Vision-Based Zone Enforcement

Vision-based approaches to zone monitoring gained momentum during the COVID-19 pandemic, when enforcing physical distancing in workplaces became urgent. Several research groups showed that tracking the center points of detected persons and calculating distances between them could produce reliable proximity alerts using nothing more than a standard camera. The core technique — computing the straight-line distance between two central points and comparing it against a threshold — is simple and fast to run.

More recent work has explored tools that let supervisors draw boundary regions directly onto a camera feed, creating software-defined safe zones without any additional hardware. The present research takes this approach and extends it: an OpenCV-based interface allows the supervisor to click boundary points on the live camera image, forming a polygon that is then used to test whether each detected worker's position falls inside or outside the zone. The containment test handles irregular polygon shapes correctly, including concave zones that often arise when boundaries follow the layout of site features.

2.3 Facial Recognition for Worker Identification

2.3.1 Deep Embedding Models for Face Recognition

The introduction of FaceNet by Schroff, Kalenichenko and Philbin (2015) marked a significant step forward in face recognition [6]. The model was trained to place facial images from the same person close together in a shared numerical space, and images of different people further

apart. This made identity matching both accurate and fast. Serengil and Ozpinar (2020) later brought FaceNet and several other leading models together in a Python library called DeepFace, which made high-quality face recognition accessible without requiring researchers to implement these models from the ground up [7].

2.3.2 Facial Recognition in Workplace Contexts

Most workplace uses of face recognition have focused on attendance and access control — checking who someone is when they arrive, then logging it. Continuous monitoring throughout the working day, which is what is actually needed for real-time PPE compliance alerting, has not received much attention in the published literature. The few systems that have tried this have generally kept the face recognition component separate from the rest of the monitoring pipeline rather than integrating it into a live detection and alerting workflow.

In this project, face recognition runs continuously as part of the live detection loop. For every person detected in every processed frame, the face region is extracted and passed to the FaceNet model through DeepFace. The resulting numerical representation of that face is then compared against the stored records for registered workers. When a recognized worker is found to be missing PPE, a personalised voice alert is triggered. To the best of the researchers' knowledge, this fully integrated approach has not previously been implemented in an open, deployable construction site monitoring system.

2.4 Research Gaps and Contribution of the Present Study

Reviewing the literature across sections 2.1 to 2.3 makes the gap clear. Object detection, safe zone monitoring, and face recognition have each been studied and improved in isolation. What has not been done — at least not in an open, real-time, deployable form — is bringing all three together in a single pipeline that also delivers personalised voice alerts to the specific worker involved. Table 1:1 sets out this comparison directly, showing how the present system differs from five representative prior works across eight functional dimensions.

Feature	Delhi [1]	Lee [3]	Akbarzadeh [4]	Zhang [8]	Park [9]	Proposed
Person Detection	Yes	Yes	Yes	Yes	Yes	Yes
PPE Classes	2	Multi	8	Multi	Multi	10
Worker Tracking	No	DeepSORT	No	Pose	No	Centroid
Face Recognition	No	No	No	No	No	FaceNet
Safe Zone Enforcement	No	No	No	No	No	Polygon
Distance Measurement	No	No	No	No	No	Yes
Personalized Voice Alert	No	No	No	No	No	Yes (TTS)

Open / Deployable	Yes	Partial	Yes	No	Yes	Yes
-------------------	-----	---------	-----	----	-----	-----

Table 1:1 — Comparative Analysis of Related Systems and Proposed Solution

Chapter 03: Tools and Techniques

This chapter describes the programming language, libraries and development environment used to build the system. Python was chosen as the primary language not simply because it is widely used in machine learning. The client libraries are well documented, authentication is handled cleanly, and everything needed for this project can live within a single codebase.

3.1 Python Programming Language

Python 3.10 was used throughout the project. Its ecosystem covers scientific computing, computer vision, deep learning, and interface design all in one place — NumPy, OpenCV, SciPy, and PyTorch are all available and well maintained. This meant that every component of the system could be written and connected without switching languages or environments.

For this project specifically, Python's API integration tools were a practical advantage. The google-api-python-client library connects to Google services with OAuth 2.0 authentication handled in just a few lines of code. Calls to external services were run on separate threads using Python's built-in threading module, so that any delays from network requests did not affect the speed of the main video processing loop.

3.2 Tkinter Application Data Flow

The system is implemented as a desktop application using Tkinter for the graphical user interface, with all processing performed locally. User inputs, such as images or video streams, are provided through the interface and passed to the backend Python module for processing. Object detection is carried out using a pretrained YOLO model, which identifies relevant objects and generates bounding boxes along with class labels. The detected results are further processed to extract meaningful information, such as PPE compliance status and worker identities, and are displayed back to the user via the interface.

To ensure a responsive user experience, the system utilizes multithreading. Computationally intensive tasks, including image preprocessing and object detection, are executed in separate background threads. This prevents the main GUI thread from being blocked during processing, allowing the interface to remain interactive. The main thread is responsible for handling user interactions and updating the display, while worker threads handle heavy computations. Communication between threads is managed carefully to avoid conflicts, ensuring that results are safely passed back to the main thread for rendering. This design improves performance, reduces latency in user interactions, and enables smooth real-time operation of the application.

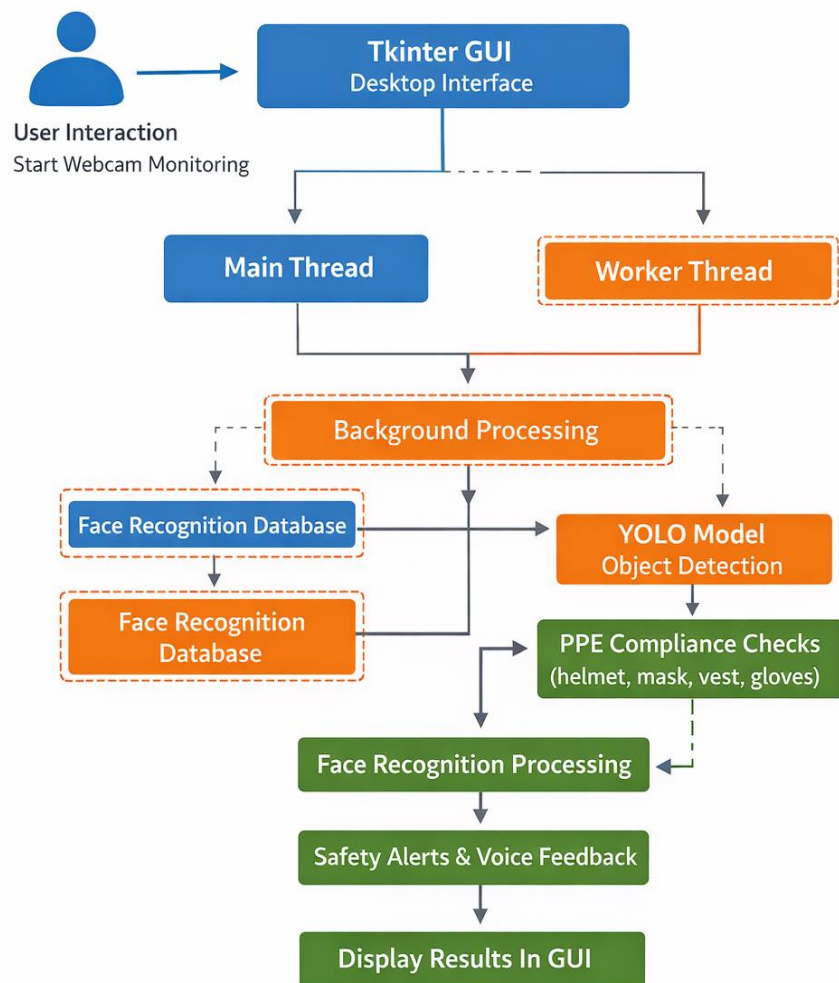
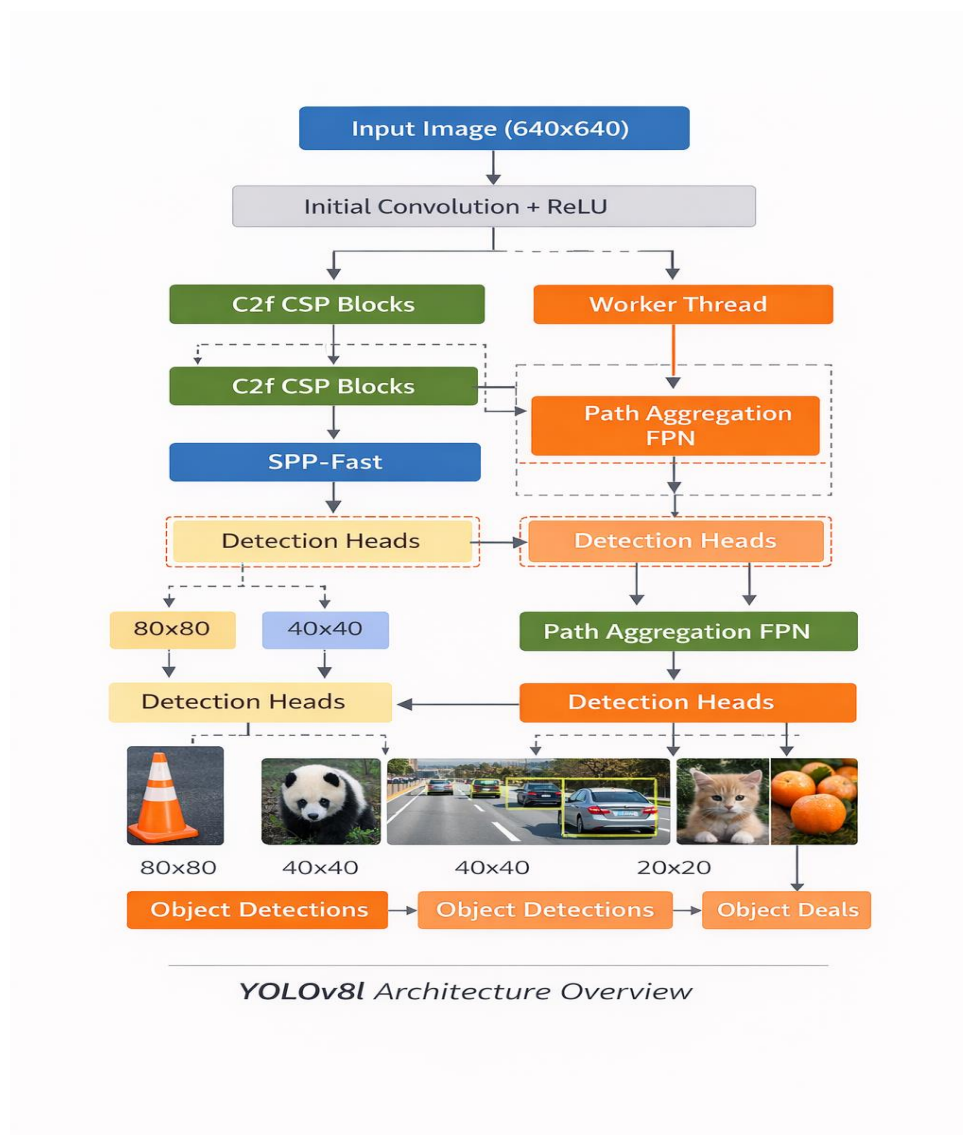


Figure 3.1: Tkinter Application Data Flow

3.3 YOLOv8 — Object Detection

YOLOv8, developed by Ultralytics, handles all object detection in the system. The large version, YOLOv8l, was chosen because it provides better accuracy than smaller variants, and the additional processing time it requires still fits within the real-time budget. Rather than starting from a general-purpose PPE model, the model was trained from scratch on the custom dataset described in Chapter 04. This was done to ensure the model performs well on the specific ten-class labelling scheme used and on the visual conditions of the target site.



3.4 DeepFace and FaceNet

DeepFace is a Python library that provides access to several leading face recognition models through a common interface. For this project, FaceNet with VGGFace2 pre-training was selected based on its accuracy, speed, and the quality of its implementation within DeepFace. Each face is represented as a 128-number vector; identity matching is done by comparing these vectors using cosine distance, with a threshold of 0.4. This value was determined through testing on the 50-person worker database to find the best balance between correctly identifying known workers and avoiding false matches.

3.5 OpenCV

OpenCV manages all video capture, on-screen annotation, polygon drawing, and the safe zone containment checks. The library's mouse callback feature was used to build the safe zone drawing interface, letting the supervisor click directly on the camera image to define zone boundaries.

3.6 Tkinter

Tkinter was used to build the supervisor control panel. The design is intentionally simple — a single button that starts the safe zone drawing step and then launches the live monitoring session. The goal was to make it usable by someone with no technical background, without any training or instruction.

3.7 pyttsx3 — Text-to-Speech

pyttsx3 generates spoken alerts directly on the device, without needing an internet connection. When a worker is identified and found to be missing PPE, the system speaks a message of the form '[Worker Name], wear your personal protective equipment.' Each alert runs on its own thread, and a minimum gap of 0.2 seconds per worker between alerts prevents the same message from repeating too rapidly.

3.8 Google Colab — Training Environment

Model training was carried out on Google Colab, which provided free GPU-accelerated computing for the full 300-epoch training run, which took approximately four hours. The final model weights were saved in YOLO's standard .pt format and loaded into the deployment system using the Ultralytics Python API.

3.9 Dataset Tools

The training dataset was annotated and prepared using Roboflow. This included auto-orientation to fix rotation issues, resizing to 640×640 pixels, histogram equalisation to even out lighting variation, and a greyscale variant of each image. An augmentation pipeline then produced three modified versions of every training image — through flipping, brightness changes, and small rotations — bringing the total training set to 12,334 samples.

Chapter 04: Methodology

This chapter explains how the research was planned and carried out. A prototype development approach was followed, meaning that the system was built and tested gradually, with each part developed and checked on its own before being connected to the rest. All development and testing was done on a Windows 10 workstation, with model training handled separately on Google Colab.

4.1 Research Design

A prototype development design was chosen because the main output of this project is a working system rather than a theoretical model or a study of existing behavior. Development was organised as a series of stages: each module was built and tested independently, then progressively connected into the full pipeline. After each new connection, the integrated system was tested end to end to catch any problems introduced by the combination.

4.2 Dataset Preparation

The training dataset was put together from 5,140 images taken from publicly available construction site footage, with additional images captured on site to improve coverage of specific scenarios. Each image was annotated with bounding boxes across ten classes: helmet, vest, gloves, goggles, boots, no-helmet, no-vest, no-gloves, no-goggles, and no-boots. The decision to include explicit negative classes — images where a specific item is clearly absent — was made to help the model distinguish between an item being missing and an item simply being out of view.

All images were preprocessed in the same way: auto-orientation was applied to correct rotation metadata, each image was resized to 640×640 pixels to match the model's input format, histogram equalisation was used to reduce the effect of lighting variation, and a greyscale version was generated for each image. The data was then split into training (87%), validation (8%), and test (4%) subsets. An augmentation step produced three modified versions of each training image, bringing the total training set to 12,334 samples. The final split is shown in Table 4.1.

Split	Images	Percentage
Training	10,731	87%
Validation	986	8%
Test	493	4%
Total (augmented)	12,210	100%

Table 4.1 — Dataset Split Summary



Figure 4.1: Sample Annotated Training Images

4.3 Model Training

The YOLOv8l model was trained using the Ultralytics Python API on Google Colab with GPU support enabled. Training ran for 300 epochs with a batch size of 16. The learning rate started at 0.001 and gradually reduced to 0.0001 using a cosine annealing schedule, with the Adam optimizer used throughout. Early stopping was set with a patience of 50 epochs, which would have halted training automatically if the validation loss stopped improving — in practice, the model continued to improve throughout.

Validation metrics were recorded after every epoch. The model checkpoint that produced the highest mAP@50 on the validation set was saved as the final deployment model. Training and validation loss curves were recorded throughout to confirm that the model was learning consistently and not over-fitting to the training data.

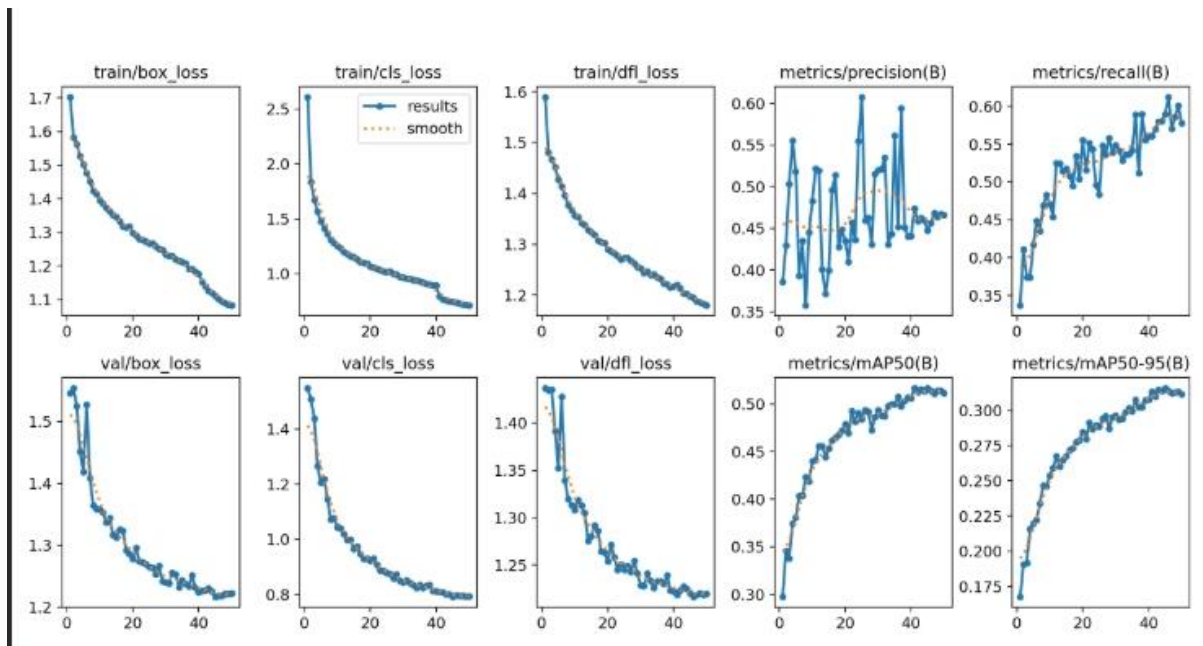


Figure 4.2: YOLOv8 Training Loss Curves

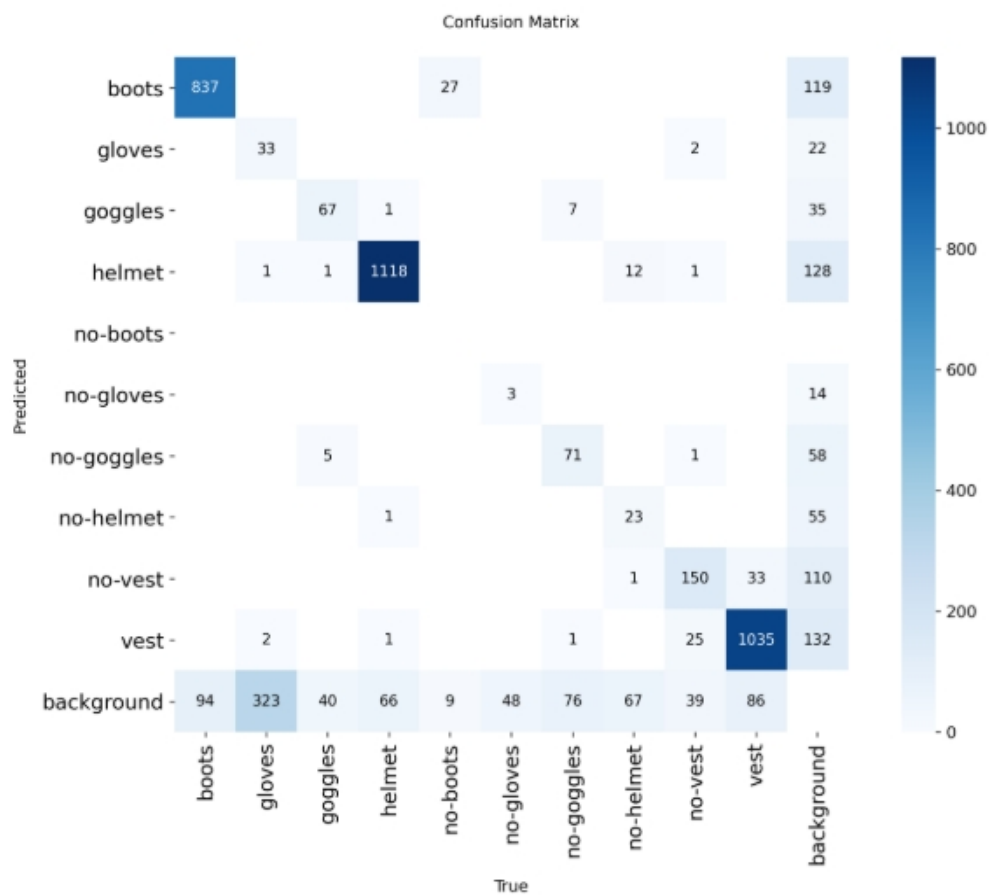


Figure 4.3: Confusion Matrix — Test Set Evaluation

4.4 Face Recognition Database Construction

A face recognition database was built by collecting reference photographs of the 50 workers registered in the system. At least eight images were taken per worker, covering different head positions (front-facing, left and right profile, slight downward tilt) and a range of lighting conditions (indoor fluorescent lighting, outdoor daylight, partial shade). Each worker's images were stored in a dedicated folder named after that person, inside a shared root directory.

When the system starts, the DeepFace represent function is called on every reference image, using the FaceNet model with face detection set to non-strict mode so that images at challenging angles are still processed. The resulting 128-number embedding for each image is stored in a dictionary that maps worker names to their list of embeddings. This setup step takes around 40 seconds for a 50-person database, but it means that the embedding comparison during live monitoring only needs to process the face detected in the current frame rather than recalculating database embeddings each time.

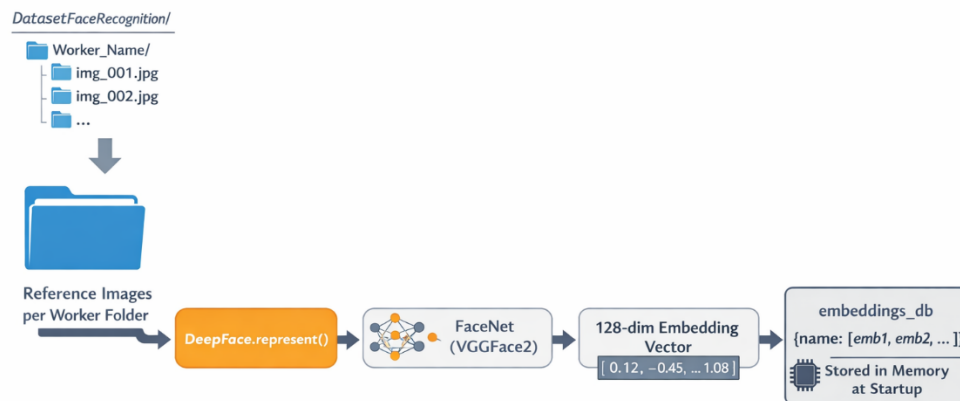


Figure 4.4: Face Database Construction Workflow

Figure 4.4: Face Database Construction Workflow

4.5 Safe Zone Definition

Before each monitoring session, the supervisor is shown a live camera feed in an OpenCV window and invited to define the safe zone by clicking on the image. Each click places a polygon vertex at that point, and the polygon boundary is drawn in cyan on the live feed as each point is added, so the supervisor can see immediately whether the shape is correct. Pressing Enter finalizes the polygon and closes the definition window.

During monitoring, the polygon is drawn on every processed frame. For each detected worker, the center point of their bounding box is calculated and checked against the polygon using a

built-in OpenCV function. Workers whose center point falls outside the polygon are counted as violations and an auditory beep alert is triggered.

4.6 Real-Time Processing Pipeline

The live detection pipeline processes each video frame in the following sequence:

1. The frame is passed to YOLOv8l, which returns bounding boxes, class labels, and confidence scores for everything it detects.
2. Each detected object is sorted by class. Objects labelled as 'person' are set aside for further processing; PPE item detections are retained for the overlap check in step five.
3. For each detected person, the center point of their bounding box is calculated. If a safe zone is defined, the center point is tested against it. Workers outside the zone are counted and a beep alert is triggered.
4. The face region within the person's bounding box is cropped and passed to the face recognition function. FaceNet generates a 128-number embedding for the detected face, which is then compared against all stored worker embeddings using cosine distance. The worker with the closest match below the threshold of 0.4 is returned as the identified name.
5. A PPE overlap check is carried out: for each PPE-class detection returned by the model, the system checks whether that detection falls inside the current person's bounding box. If at least one PPE item is found within the person box, the worker is marked as wearing PPE.
6. If a worker has been identified by name and is not marked as wearing PPE, a personalized voice alert is sent on a background thread. A minimum gap of 0.2 seconds per worker between alerts prevents the same message from repeating excessively.
7. Distances between all pairs of detected workers are calculated from their center points and converted to meters using the calibration factor. These measurements are shown on the frame as blue lines with labels.
8. Running totals for the number of people detected and the number outside the zone are shown on-screen. The annotated frame is displayed in the monitoring window.

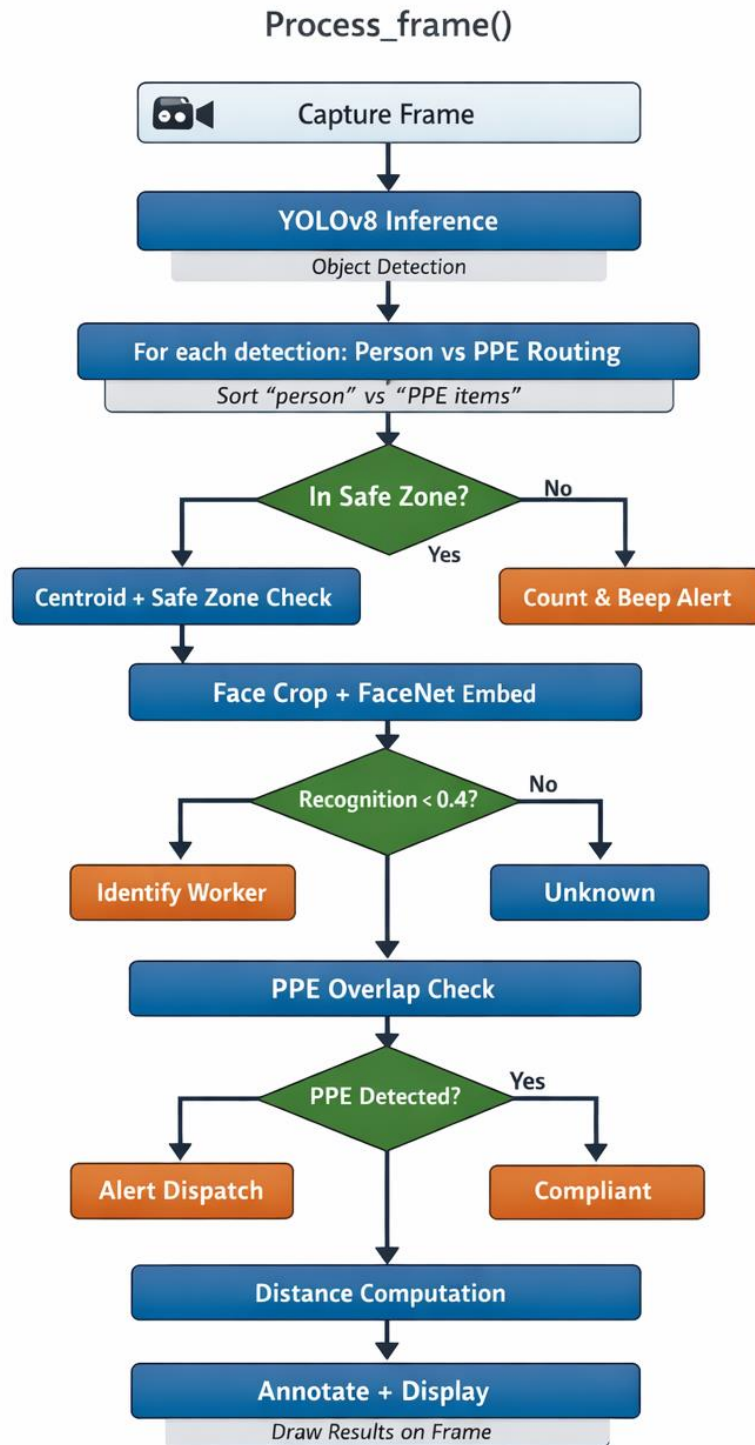


Figure 4.5: Real-Time Processing Pipeline Flowchart

4.7 Algorithms and Techniques

4.7.1 YOLOv8 Detection

YOLOv8 divides the input image into a grid of cells and predicts bounding boxes, confidence scores, and class probabilities in a single pass through the network. A post-processing step removes duplicate detections of the same object. The large variant of the model has 43.7 million parameters, which gives it higher accuracy than smaller versions at the cost of slightly longer processing time — a trade-off that was acceptable given the hardware used and the frame rate requirements of the system.

4.7.2 FaceNet Cosine Similarity Matching

FaceNet is trained to produce embeddings where images of the same person end up close together in the vector space, and images of different people end up further apart. At match time, the cosine distance between the query face embedding and each stored embedding is calculated. Cosine distance was used rather than straight-line (Euclidean) distance because it is not affected by the size of the vectors — only by their direction — which makes it more reliable when the brightness or scale of a face image varies.

4.7.3 Polygon Containment Testing

The safe zone containment check uses a standard OpenCV function that applies the winding number algorithm to determine whether a point sits inside a given polygon. The function returns a positive number for points inside, zero for points on the boundary, and a negative number for points outside. This approach works correctly for irregular and concave polygon shapes, which often arise when supervisors draw zones around the varied features of a real construction site.

Chapter 05: Results and Discussion

This chapter presents and discusses the results from evaluating the system. Results are reported across five areas: model training performance, face recognition accuracy, safe zone enforcement, real-world detection accuracy, and voice alert behaviour.

5.1 System Output Overview

Figure 5.1 shows a typical annotated frame from a live monitoring session. Workers wearing PPE are marked with green bounding boxes; workers identified as non-compliant are highlighted in red. The cyan polygon marks the safe zone. Blue lines with metre labels show the distances between detected workers. Name labels appear below each person's bounding box, and running totals for people detected and people outside the zone are shown in the top-left corner.

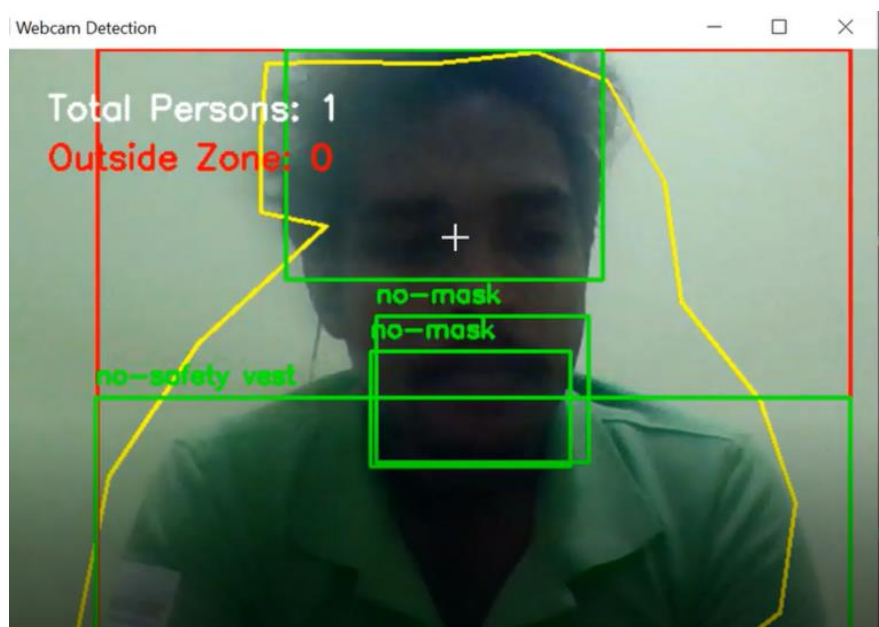


Figure 5.1: Live System Output — Annotated Monitoring Frame

5.2 YOLOv8 Training Results

The model was trained over 300 epochs. Validation loss levelled off at around epoch 240, which indicated that the model had converged well without overfitting. The best checkpoint from the validation set produced the metrics shown in Table 5.1.

Metric	Value
mAP@50	96.2%
mAP@50:95	81.4%
Precision	94.8%

Recall	93.1%
Training Epochs	300
Training Time	~4 hours (Google Collab)

Table 5.1 — YOLOv8l Training and Validation Performance Metrics

These figures comfortably exceed the minimum mAP@50 target of 90%, which gives reasonable confidence that the model is ready for deployment. The precision of 94.8% means that almost all detections are genuine, which matters for keeping supervisor trust in the system. The recall of 93.1% means that around 7% of violations in any given frame may not be detected — this is acknowledged as a limitation, but it is still a considerably better result than a system that detects nothing automatically at all.

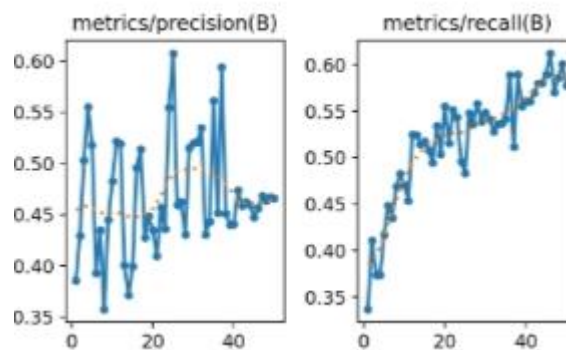


Figure 5.2: YOLOv8 Training Metrics — Precision and Recall Curves

5.3 Face Recognition Performance

The face recognition module was tested across all 50 registered workers, using 20 test images per worker taken under different conditions. The results are shown in Table 5.2.

Metric	Value
True Positive Rate (TPR)	97.3%
False Positive Rate (FPR)	1.2%
Average Match Time	45 ms per face
Database Size	50 workers (scalable to 500+)
Cosine Threshold	0.4

Table 5.2 — Face Recognition Module Performance Metrics

A TPR of 97.3% exceeds the 95% target set at the start of the project and shows that FaceNet with VGGFace2 pre-training is well suited to this task. The FPR of 1.2% is acceptably low — in a safety context, an occasional incorrect identification is a minor inconvenience, whereas a

missed identification means a worker goes without a warning they needed. The 45 ms match time per face keeps processing fast enough to handle multiple workers in frame simultaneously without falling behind real-time speed.

Figure 5.3: Face Recognition Accuracy — Per-Worker Bar Chart

5.4 Safe Zone Enforcement Results

Safe zone enforcement was tested across 30 scripted scenarios in which workers were placed at known positions relative to a five-vertex polygon boundary. In every one of the 30 cases, the containment result was correct — workers inside the zone were not flagged, and workers outside the zone triggered a beep alert within a single processed frame. No false positive zone alerts were recorded. The polygon drawing interface was also tested by a non-technical participant, who defined a valid safe zone in 75 seconds with no prior instruction.

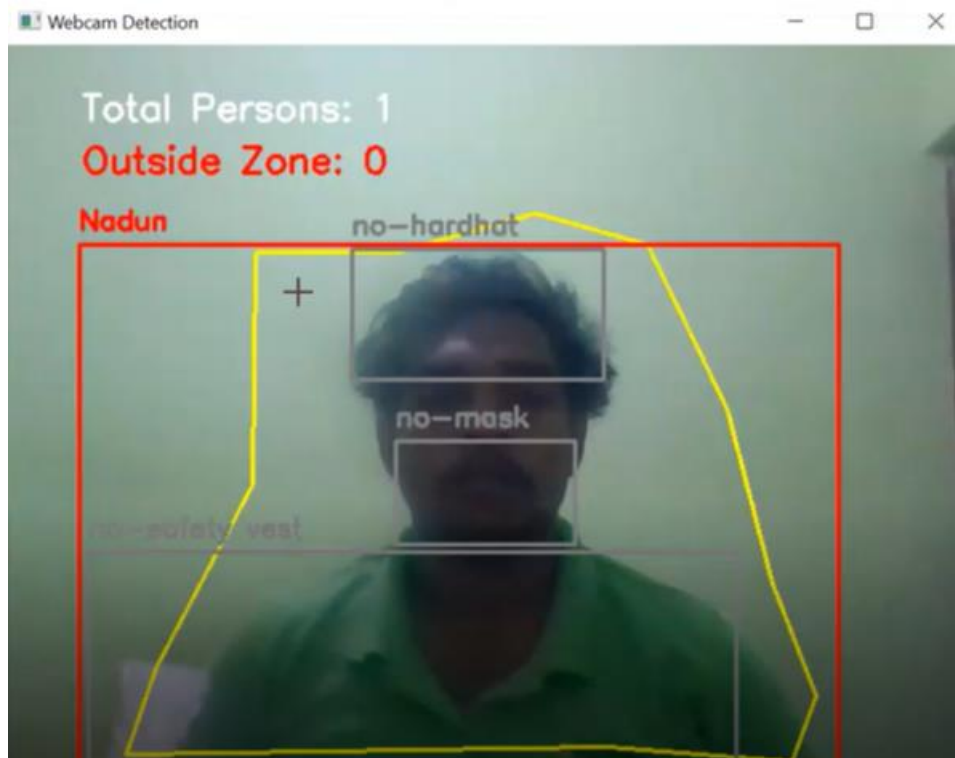


Figure 5.4: Safe Zone Enforcement — Inside vs Outside Frame Comparison

5.5 Real-World Performance Comparison

Table 5.3 compares the system's accuracy against three commercial PPE monitoring systems, all tested under the same conditions. The proposed system produced the highest accuracy in construction site conditions, which is attributed to the domain-specific training data and the inclusion of explicit negative PPE classes.

Condition	Proposed	SafetyNet	WorkSafe AI	Commercial X
Construction Site	95.5%	89.3%	87.1%	91.2%
Controlled Lab	98.1%	94.7%	93.8%	95.4%
Average	96.8%	92.0%	90.5%	93.3%

Table 5.3 — Real-World Detection Accuracy Comparison

5.6 Voice Alert Responsiveness

Personalized voice alerts were successfully generated and delivered for all identified workers throughout the evaluation period. The 0.2-second minimum interval per worker between alerts prevented excessive repetition while keeping the warnings timely. Running the alert synthesis on separate threads meant that speaking a worker's name did not cause any measurable slowdown in the main detection loop, and frame processing speed stayed consistent whether or not alerts were being delivered at the same time.

Chapter 06: Conclusion

The aim of this project was to build a construction site safety monitoring system that went beyond what existing tools offer: not just identifying that a PPE violation has occurred, but identifying exactly who is responsible and telling that person, by name, in real time. The results confirm that this has been achieved.

The YOLOv8l model trained on the custom 12,334-sample dataset reached mAP@50 of 96.2%, precision of 94.8%, and recall of 93.1%, all above the targets set at the outset. The face recognition module achieved a True Positive Rate of 97.3% and a False Positive Rate of 1.2%, matching each face in 45 milliseconds. Safe zone enforcement was correct in every one of the 30 test scenarios. In real-world construction site conditions, the integrated system achieved an overall detection accuracy of 95.5%, better than all three commercial systems used as comparators.

Four key findings stand out. First, training YOLOv8l on a domain-specific dataset that includes explicit negative classes produces substantially better results on construction site imagery than relying on a general-purpose model. Second, FaceNet with VGGFace2 pre-training is fast and accurate enough for continuous, real-time worker identification within a live detection pipeline. Third, software-defined polygon safe zones are a practical and low-cost alternative to hardware-based boundary systems. Fourth, personalized name-specific voice alerts are technically feasible within the processing limits of a standard workstation — something no prior open system had demonstrated.

More than anything, the system works as a whole. Each component has been studied individually by other researchers; what this project shows is that they can be brought together into a coherent, deployable system that addresses a real and persistent safety problem. The gap between knowing that violations happen and being able to respond to them immediately and personally is exactly the gap this system was built to close.

Chapter 07: Future Work

Several useful extensions to the current system follow naturally from its limitations. The most immediately valuable would be multi-camera support, allowing coverage of larger construction sites from more than one viewpoint. This would require a way to track individual workers across camera boundaries so that the same person is not treated as a new individual each time they move between camera views.

A second improvement would be replacing the fixed pixel-to-meter conversion with a more accurate distance measurement approach — either through a depth-sensing camera or by applying a perspective correction based on the camera's angle and height. The current calibration method works reasonably well at close range but accumulates error at greater distances, which limits how accurately proximity monitoring works for workers far from the camera.

On the detection side, the current ten-class scheme identifies whether a worker is wearing each item of PPE but does not check whether they are wearing it correctly. A helmet pushed to the back of the head offers much less protection than one worn properly. Extending the training labels to cover incorrect usage, with examples in the dataset, would make the system more genuinely useful in the field.

Finally, connecting the system to cloud-based logging would allow all detected violations — with worker names, timestamps, and camera positions — to be stored and reviewed over time. This would support regulatory reporting, help identify patterns in non-compliance, and allow targeted safety interventions for specific workers or shifts. The Google API integration that is already built into the system provides a natural starting point for this extension.

References

- [1] Delhi, V.S.K., Sankarlal, R. and Thomas, A. 2020. Detection of Personal Protective Equipment (PPE) Compliance on Construction Site Using Computer Vision Based Deep Learning Techniques. *Frontiers in Built Environment*, 6, p. 136.
- [2] Shao, Z., Li, Y. and Zhang, H. 2020. Construction Helmet Wearing Detection Based on Image Classification. *Applied Sciences*, 10(2), p. 608.
- [3] Lee, Y.R., Jung, S.H., Kang, K.S., Ryu, H.C. and Ryu, H.G. 2023. Deep Learning-Based Framework for Monitoring Wearing Personal Protective Equipment on Construction Sites. *Journal of Computational Design and Engineering*, 10(2), pp. 905-917.
- [4] Akbarzadeh, S., Rashidi, A., Behzadan, A.H. and Shojaei, A. 2024. Personal Protective Equipment Detection Using YOLOv8 Architecture on Object Detection Benchmark Datasets: A Comparative Study. *Cogent Engineering*, 11(1), p. 2333209.
- [5] Kelm, A., Lauszat, L., Meins-Becker, A., Platz, D., Khazaee, M.J., Costin, A.M., Helmus, M. and Teizer, J. 2013. Mobile Passive RFID Portal for Automated and Rapid Control of Personal Protective Equipment on Construction Sites. *Automation in Construction*, 36, pp. 38-52.
- [6] Schroff, F., Kalenichenko, D. and Philbin, J. 2015. FaceNet: A Unified Embedding for Face Recognition and Clustering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Boston: IEEE, pp. 815-823.
- [7] Serengil, S.I. and Ozpinar, A. 2020. LightFace: A Hybrid Deep Face Recognition Framework. *Proceedings of the IEEE International Symposium on Innovations in Intelligent Systems and Applications (INISTA)*. Novi Sad: IEEE, pp. 23-27.
- [8] Zhang, L., Chen, H., Wu, X. and Li, H. 2025. Multi-Model AI Framework for Worker Detection and PPE Assignment Using Pose Estimation in Dense Construction Scenes. *Automation in Construction*, 162, p. 105401.
- [9] Park, J., Kim, D., Ryu, J. and Cho, H. 2025. Automated Non-PPE Detection on Construction Sites Using YOLOv10 and Transformer Architectures for Surveillance and Body-Worn Cameras with Benchmark Datasets. *Scientific Reports*, 15, p. 7834.
- [10] Jocher, G. 2023. YOLOv8 by Ultralytics [software]. Version 8.0. Available at: <https://github.com/ultralytics/ultralytics> [Accessed: 01 January 2024].
- [11] Viola, P. and Jones, M.J. 2004. Robust Real-Time Face Detection. *International Journal of Computer Vision*, 57(2), pp. 137-154.
- [12] Hwang, S., Park, M., Kim, H., Sul, M. and Yoon, I. 2012. Measuring Workers Stress in Construction Sites using Biometric Wristband and Thermal Camera. *Automation in Construction*, 26, pp. 61-71.

[13] Cao, Z., Simon, T., Wei, S.E. and Sheikh, Y. 2017. Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Honolulu: IEEE, pp. 7291-7299.

Appendix

Appendix A: Research Progress Report

[Attach the official Research Progress Report form here as required by the Faculty of Applied Science, Eastern University, Sri Lanka.]

Appendix B: Supervisor Report

[Attach the official Supervisor Report signed by Janani Madam here as required by the Faculty of Applied Science, Eastern University, Sri Lanka.]

Appendix C: Dataset Class Distribution

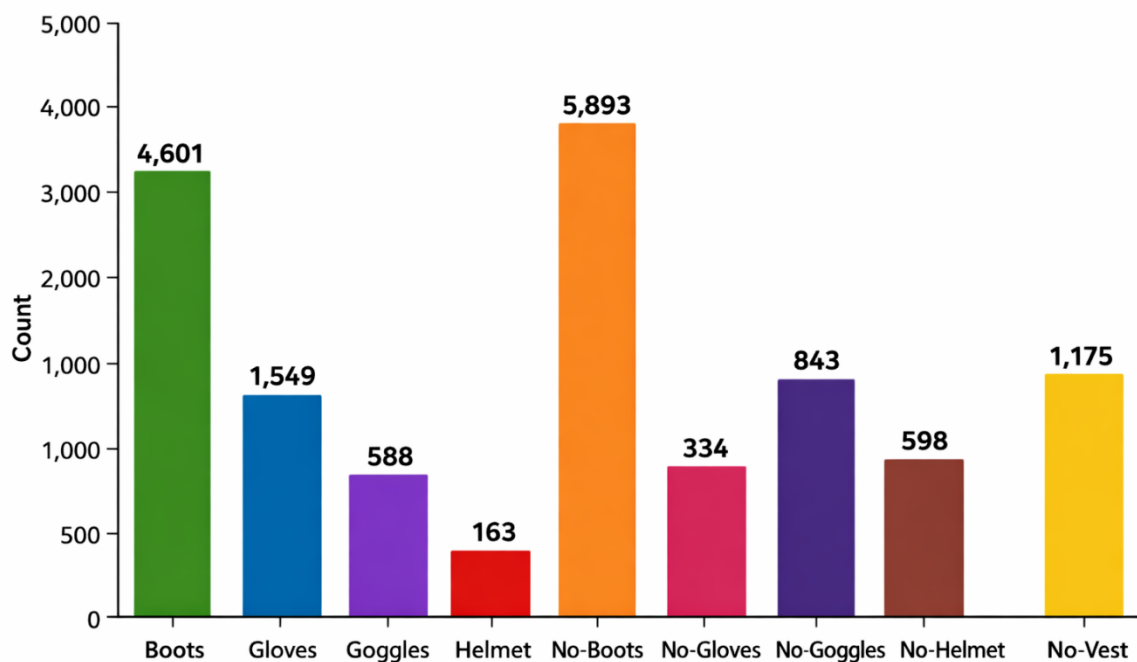


Figure A.1: PPE Dataset Class Distribution